NAME:**S.KARTHIKEYAN** **3122 22 5001 056**

CSE – B DATE: **15.12 .2023**

CAT 2 ASSIGNMENT

NON-LINEAR DATA STRUCTURE

CONCEPT:

In this question, a house can be built only if its predecessor has been built. One of the rooms is already built and is Room 0. Room 0 has a prevRoom value of -1 while others have value of predecessor vertex.

So topological sort can be applied because i) graph is directed

ii) graph doesn't have cycles

Since, Topological Ordering doesn't give us unique orders, thus leading to different combinations.

Apart from Topological Sorting, DFS and BFS are applied to get different combinations. For BFS, Queue ADT is used and for DFS, Stack ADT is used.

CODE:

antsadt.h

```c
#include <stdio.h>
#include <stdlib.h>

struct graph
{
    int n;
    int adj[100][100];
};

void init(struct graph *g,int n)
{
    g->n=n;
    for (int i=0;i<n;i++)
    {
        for (int j=0;j<n;j++)
        {
            g->adj[i][j]=0;
        }

    }
}
```

NAME:**S.KARTHIKEYAN** **3122 22 5001 056**
CSE – B DATE: **15.12 .2023**

```c
void construct(struct graph *g,int arr[])
{

    for (int i=1;i<g->n;i++)
    {
        g->adj[arr[i]][i]=1;
    }

}

void display(struct graph *g)
{
    for (int i=0;i<g->n;i++)
    {
        for (int j=0;j<g->n;j++)
        {
            printf("%d ",g->adj[i][j]);
        }
        printf("\n");
    }
}



struct queue
{
    int arr[100];
    int f,r;
    int size;
};


void initqueue(struct queue *h,int n)
{
    h->size=n;
    h->f=h->r=-1;
}



void enqueue(struct queue *h,int ele)
{
    h->r+=1;
        h->arr[h->r]=ele;
}
int isempty(struct queue *h)
{
    if (h->f==h->r)
        return 1;
```

```
    else
        return 0;
}
int dequeue(struct queue *h)
{
    h->f+=1;
    int d=h->arr[h->f];
    return d;
}

void toposort(struct graph *g,struct queue *q)
{

    int indeg[g->n];
    for (int i=0;i<g->n;i++)
    {
        int c=0;
        for (int j=0;j<g->n;j++)
        {
            if (g->adj[j][i]==1)
                c++;
        }
        indeg[i]=c;
    }

    for (int i=0;i<g->n;i++)
    {
        if (indeg[i]==0)
            enqueue(q,i);
    }

    while (!isempty(q))
    {
        int z=dequeue(q);
        printf("%d --> ",z);
        for (int i=g->n;i>=0;i--)
        {
            if (g->adj[z][i]==1 && --indeg[i]==0)
            {
                enqueue(q,i);
            }
        }
    }
    printf("end");
}

void bfs(struct graph *g,struct queue *q,int start)
{
```

```
    int visited[g->n];
    for (int i=0;i<g->n;i++)
    {   visited[i]=0;   }
    enqueue(q,start);
    visited[0]=1;
    while (!isempty(q))
    {
        int z = dequeue(q);
        printf("%d --> ",z);
        for (int i=0;i<g->n;i++)
        {
            if (g->adj[z][i]==1 && visited[i]!=1)
            {
                visited[i]=1;
                enqueue(q,i);

            }

        }

    }
    printf("end");

}
struct stack
{   int size;
    int top;
    int arr[100];
};

void createstack(struct stack *s,int size)
{
    s->size=size;
    s->top=-1;
}



void push(struct stack *s,int ele)
{
     s->top+=1;
    s->arr[s->top]=ele;
}

int isemptystack(struct stack *s)
{
    if (s->top==-1)
        return 1;
    else
```

```
        return 0;
}

void popstack(struct stack *s)
{
    s->top-=1;
}

int peek(struct stack *s)
{
     return s->arr[s->top];
}

void dfs(struct graph *g,struct stack *s1,int start)
{
    int visited[g->n];
    for (int i=0;i<g->n;i++)
    {
        visited[i]=0;
    }
    push(s1,start);
    visited[start]=1;
    printf("%d --> ",start);
    while (!isemptystack(s1))
    {
        int t=peek(s1);
        int flag =0;
        for (int i=0;i<g->n;i++)
        {
            if (g->adj[t][i]==1 && visited[i]!=1)
            {
                flag =1;
                visited[i]=1;
                push(s1,i);
                printf("%d --> ",i);
                break;
            }

        }
        if (flag == 0)
            popstack(s1);
    }
    printf("end");
}

void dfs_neg(struct graph *g,struct stack *s1,int start)
{
    int visited[g->n];
```

```
    for (int i=0;i<g->n;i++)
    {
        visited[i]=0;
    }
    push(s1,start);
    visited[start]=1;
    printf("%d --> ",start);
    while (!isemptystack(s1))
    {
        int t=peek(s1);
        int flag =0;
        for (int i=g->n;i>=0;i--)
        {
            if (g->adj[t][i]==1 && visited[i]!=1)
            {
                flag =1;
                visited[i]=1;
                push(s1,i);
                printf("%d --> ",i);
                break;
            }

        }
        if (flag == 0)
            popstack(s1);
    }
    printf("end");
}
```

ants.c

```
#include <stdio.h>
#include <stdlib.h>
#include "antsadt.h"


void main()
{
    struct graph *g;
    g=(struct graph*)malloc(sizeof(struct graph));
    printf("enter the build array :\n");
    printf("enter the length :");
    int n;scanf("%d",&n);
    int prevRoom[n];
```

```
      for (int i=0;i<n;i++)
    {
        printf("enter the num of previous room of room %d :",i);
        int a;scanf("%d",&a);
        prevRoom[i]=a;
    }
    struct queue *q;
    q=(struct queue*)malloc(sizeof(struct queue));
    initqueue(q,n);
    init(g,n);
    construct(g,prevRoom);
    printf("\nDISPLAYING THE ADJACENCY MATRIX\n");
    display(g);
    printf("\n SOME OF THE POSSIBILITIES\n");

    toposort(g,q);
    printf("\n");

    struct queue *q2;
    q2=(struct queue*)malloc(sizeof(struct queue));
    initqueue(q2,n);
    bfs(g,q2,0);
    printf("\n");

    struct stack *s;
    s=(struct stack *)malloc(sizeof(struct stack));
    createstack(s,n);
    dfs(g,s,0);
    printf("\n");
    struct stack *s1;
    s1=(struct stack *)malloc(sizeof(struct stack));
    createstack(s1,n);
    dfs_neg(g,s1,0);
}
```

NAME:**S.KARTHIKEYAN** **3122 22 5001 056**

CSE – B DATE: **15.12 .2023**

OUTPUT:

```
C:\Users\User\OneDrive\SEM3\DATA STRUCTURES\LAB\CAT_ASSI
enter the build array :
enter the length :5
enter the num of previous room of room 0 :-1
enter the num of previous room of room 1 :0
enter the num of previous room of room 2 :0
enter the num of previous room of room 3 :1
enter the num of previous room of room 4 :2

DISPLAYING THE ADJACENCY MATRIX
0 1 1 0 0
0 0 0 1 0
0 0 0 0 1
0 0 0 0 0
0 0 0 0 0

 SOME OF THE POSSIBILITIES
0 --> 2 --> 1 --> 4 --> 3 --> end
0 --> 1 --> 2 --> 3 --> 4 --> end
0 --> 1 --> 3 --> 2 --> 4 --> end
0 --> 2 --> 4 --> 1 --> 3 --> end
```